Flyweight

Season: 2 · Structural · Category: GoF · Level: Intermediate · Build with Waqar

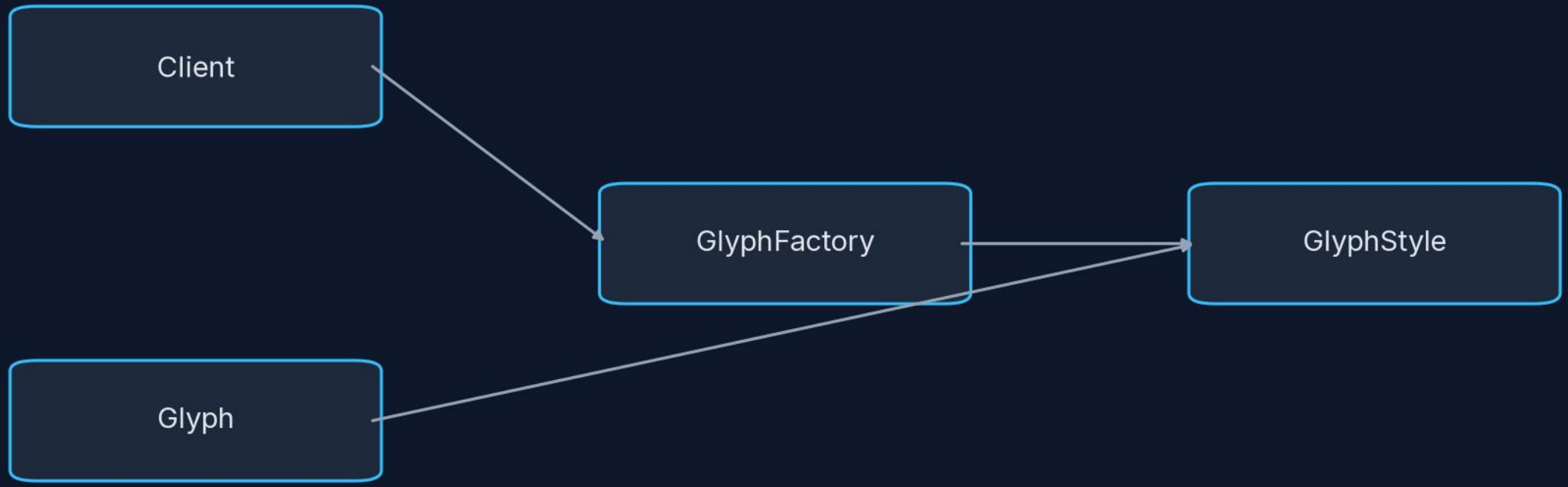
Today's Hook

- Today: Flyweight as a StreamForge product force — not trivia.
- Runnable Demo on JDK 25 with captured stdout.
- Classic encoding → Java 25 / Zazr modernization → trade-offs.
- CapCut slides + full spoken script included.

The Problem

- Naive local fixes: more ifs, statics, nulls, or threads.
- Illegal states still compile; tests become order-dependent.
- Design reviews lack a shared name for the force.
- Interviews punish memorization without judgment.

Structure



Flyweight structure

Real-World Scenario

- StreamForge creator platform is the running metaphor.
- Same names appear in naive code, refactor, and Demo output.
- Stay in one story for the full 10–12 minutes.
- Product language first; pattern vocabulary second.

Naive Code

```
// Naive sketch for Flyweight
public class Naive {
  // duplicated logic / leaked details / unsafe sharing
  Object doIt(Object... args) {
    if (args == null) return null;
    return args[0];
  }
}
```

Why Naive Hurts

- Coupling spreads across controllers and infrastructure.
- Tests need sleeps or static reset hooks.
- New variants require shotgun edits.
- On-call cannot reason about failure modes.

Implementation

```java
// file: src/main/java/com/buildwithwaqar/patterns/gof/structural/flyweight/GlyphFactory.java
package com.buildwithwaqar.patterns.gof.structural.flyweight;

import java.util.Map;
import java.util.concurrent.ConcurrentHashMap;

record GlyphStyle(String font, int size, String color) {}

record Glyph(char character, GlyphStyle style, int x, int y) {}

/** Flyweight factory pools intrinsic glyph styles. */
public final class GlyphFactory {
    private final Map<GlyphStyle, GlyphStyle> pool = new ConcurrentHashMap<>();

    public GlyphStyle style(String font, int size, String color) {
        var key = new GlyphStyle(font, size, color);
        return pool.computeIfAbsent(key, k -> k);
    }

    public Glyph glyph(char c, GlyphStyle style, int x, int y) {
        return new Glyph(c, style, x, y);
    }
```

Implementation (2)

```
    public int pooled() { return pool.size(); }
}
```

Demo.main

```
package com.buildwithwaqar.patterns.gof.structural.flyweight;
public final class Demo {
  public static void main(String[] a) {
    System.out.println("=== Flyweight walkthrough ===");
    var f = new GlyphFactory();
    var s1 = f.style("Inter", 14, "#e2e8f0");
    var s2 = f.style("Inter", 14, "#e2e8f0");
    var s3 = f.style("Inter", 18, "#e2e8f0");
    System.out.println("sameStyle=" + (s1 == s2) + " pooled=" + f.pooled());
    System.out.println("glyph=" + f.glyph('W', s1, 10, 20));
    System.out.println("thirdDistinct pooled=" + f.pooled());
  }
}
```

Walkthrough Output

```
=== Flyweight walkthrough ===
sameStyle=true pooled=2
glyph=Glyph[character=W, style=GlyphStyle[font=Inter, size=14, color=#e2e8f0], x=10, y=20]
thirdDistinct pooled=2
```

Step-by-Step Trace

- Each stdout line maps to a refactor stage.
- Re-run Demo locally on JDK 25 to verify.
- Differences usually mean a dirty tree — pull main.
- Pause the video and predict the next line before revealing.

Modern Java 25 / Zazr

```
// Java 25 / Zazr direction for Flyweight
// records + sealed + switch
// Option/Either/Validation instead of null
// virtual threads for blocking fan-out
// inject interfaces at the edge
```

Trade-offs

- Clarity vs ceremony — do not spawn five types for two lines.
- Closed sealed worlds vs open SPI plugins.
- Teaching demos omit production idempotency/metrics.
- Prefer simpler language features when they erase the force.

Related Patterns

- Contrast with the neighbor pattern (force difference).
- Name the roles from memory in sixty seconds.
- Point to the season map for what comes next.
- Link the GitHub episode folder in the description.

Recap

- Force → naive pain → Flyweight → Demo stdout.
- Modernize with Java 25 / Zazr when it shrinks code.
- Teach when NOT to use it.
- Practice: break a test, fix it, re-run Demo.